

Assignment3

Info

Name: Cody (Zhexian) Liu

Assignment Number: 3

Repo Link: <https://github.com/Rorical/CWM-FDI>

Simple Measurements - Idleness

1. Top consumers of CPU and MEM resources

```
ubuntu@ubuntu:~/CWM-FDI$ ps
  PID TTY          TIME CMD
 28945 pts/5    00:00:00 bash
 28985 pts/5    00:00:00 ps
```

```
ubuntu@ubuntu:~/CWM-FDI$ ps aux
USER                PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
...
ubuntu             5832  3.0  4.1 12411184 671124 ?        Sl   09:04   3:32 firefox
ubuntu            10789  1.1  2.0 1520670520 329800 ?        Ssl  09:59   0:40 code
ubuntu            11160 12.7  2.4 1522299112 392956 ?        Sl   09:59   7:38 code
render
ubuntu            16120  6.0  2.0 4665488 334252 ?        Ssl  10:10   2:53 gnome-
shell
...
```

Top CPU consumers: code renderer (12.7%).

Top MEM consumers: firefox (4.1%).

To be a top user, one could run CPU-intensive workloads like code compilation or keep many browser tabs and applications open to consume memory.

2. Overall idle percentage

The machine's Busy% ranges from **22–35%**, meaning **65–78% idle**. Apart from the desktop environment, browser, and VS Code, most CPU cores spend the majority of their time in idle state.

Resource Usage – Compute Power, Energy, and Thermals

3. Baseline fluctuation over time

```
ubuntu@ubuntu:~/CWM-FDI$ sudo turbostat -q -S --show PkgWatt -i 1
```

PkgWatt

9.01
7.71
3.88
5.03
5.50
3.33
4.33
3.79
3.87
3.25
3.34
3.30
3.26
4.03
3.48
3.95
10.10
3.48
3.44

```
ubuntu@ubuntu:~/CWM-FDI$ sudo turbostat -q -S --Joules --show Pkg_J -i 1
```

Pkg_J

2.31
2.67
3.42
2.48
2.37
2.83
3.03
3.09
2.82
4.18
4.71
4.72
2.95
2.98
2.32
2.26
2.37

2.28

2.36

Yes, the baseline fluctuates over time. Spikes occur when background services or periodic OS tasks, interruptions from external devices briefly wake the CPU. Background processes such as tracker-miner, updates services cause intermittent power bursts.

4. Baseline Busy%, MHz, power, temperature

```
ubuntu@ubuntu:~/CWM-FDI$ sudo turbostat -q -S --show
```

```
Busy%,Bzy_MHz,PkgWatt,CorWatt,PkgTmp,CoreTmp -i 1
```

Busy%	Bzy_MHz	CoreTmp	PkgTmp	PkgWatt	CorWatt
35.26	2393	38 38	8.04	5.30	
31.36	1220	36 37	3.90	1.30	
31.45	1555	36 38	5.02	2.38	
30.59	1465	36 37	5.50	1.98	
23.56	1232	36 37	3.35	1.14	
22.54	1650	36 37	4.31	2.00	
27.40	1319	37 37	3.83	1.43	
24.28	1416	36 37	3.84	1.47	
23.58	1222	37 37	3.27	1.02	
24.79	1218	36 37	3.33	1.05	
23.03	1225	37 37	3.29	1.06	
22.44	1222	36 37	3.29	1.02	
25.68	1381	35 37	4.02	1.58	
28.20	1229	37 37	3.51	1.14	
28.54	1245	36 36	3.61	1.21	
33.17	3068	36 37	10.40	7.77	
22.78	1388	36 37	3.50	1.18	
24.70	1306	36 37	3.45	1.12	
29.03	1305	36 37	3.65	1.31	

Busy%: 22–35% → 65–78% idle. This is consistent with observation from ps and htop.

turbostat provides a more detailed view of idleness than ps/htop. It quantifies the exact percentage of time the CPU is actively executing vs idle. We can also see brief spikes during idle times.

5. Baseline sleep states

```

ubuntu@ubuntu:~/Documents/CWM-FDI$ sudo turbostat -q -S --show Pkg%pc2,Pkg%pc3,Pkg%pc6,Pkg%pc7,Pkg%pc8,CPU%c1,CPU%c3,CPU%c6,CPU%c7 -i 1
CPU%c1 CPU%c3 CPU%c6 CPU%c7 Pkg%pc2 Pkg%pc3 Pkg%pc6 Pkg%pc7 Pkg%pc8
1.26 0.08 5.67 90.94 16.02 73.32 0.00 0.00 0.00
2.21 0.20 6.99 85.56 16.64 63.52 0.00 0.00 0.00
5.72 1.12 14.19 63.42 19.29 16.98 0.00 0.00 0.00
4.00 0.59 14.17 68.91 21.15 15.40 0.00 0.00 0.00
3.19 0.44 13.52 72.45 22.49 19.77 0.00 0.00 0.00
2.44 0.21 11.84 76.52 23.94 20.50 0.00 0.00 0.00
2.45 0.21 13.47 75.18 25.13 20.18 0.00 0.00 0.00
2.30 0.17 11.96 76.99 23.74 21.29 0.00 0.00 0.00
4.89 0.70 14.15 66.46 22.15 11.92 0.00 0.00 0.00
5.56 1.11 18.37 53.83 17.09 10.03 0.00 0.00 0.00
5.01 1.07 15.09 59.03 18.41 12.61 0.00 0.00 0.00
4.98 1.07 11.00 67.39 16.99 36.65 0.00 0.00 0.00
4.55 0.91 14.00 66.70 16.95 32.41 0.00 0.00 0.00
4.61 0.61 13.41 61.67 13.73 25.64 0.00 0.00 0.00
4.07 0.87 9.46 74.74 14.79 52.94 0.00 0.00 0.00
3.70 0.72 8.53 78.43 15.78 55.41 0.00 0.00 0.00
2.69 0.36 7.86 83.30 15.64 62.96 0.00 0.00 0.00
2.63 0.44 7.10 80.54 15.03 58.60 0.00 0.00 0.00
3.72 0.64 12.29 72.77 21.21 32.04 0.00 0.00 0.00
5.21 1.13 14.89 53.39 14.24 15.89 0.00 0.00 0.00

```

The CPU spends significant time in shallow-to-medium sleep states. No time is spent in deep package sleep states.

6. stress-ng observations

6a. Energy during stress test

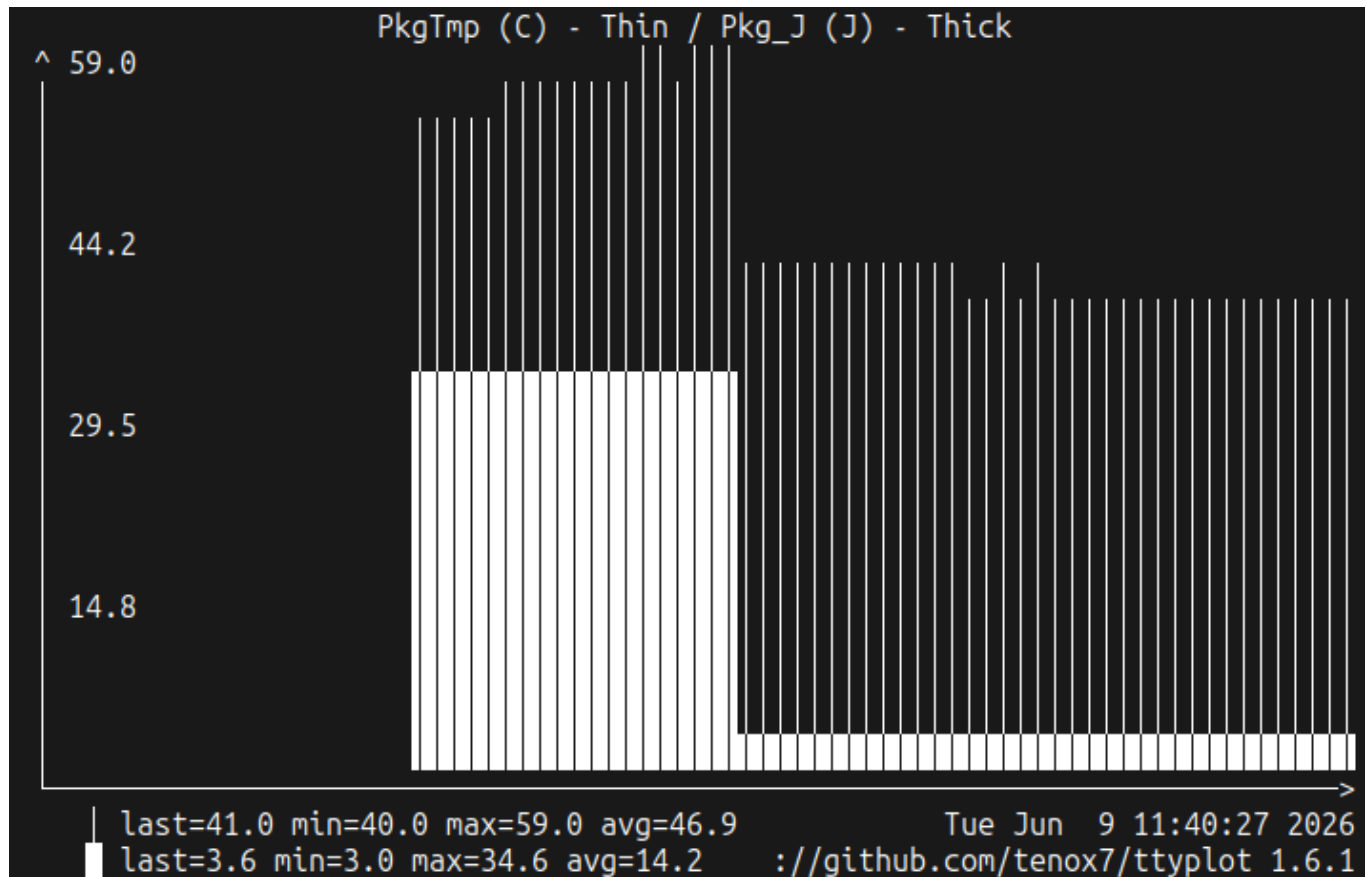
```

ubuntu@ubuntu:~$ sudo turbostat -q -S --Joules --show Pkg_J stress-ng -c 4 -
-cpu-method matrixprod -t 20s
stress-ng: info: [34691] setting to a 20 secs run per stressor
stress-ng: info: [34691] dispatching hogs: 4 cpu
stress-ng: info: [34691] skipped: 0
stress-ng: info: [34691] passed: 4: cpu (4)
stress-ng: info: [34691] failed: 0
stress-ng: info: [34691] metrics untrustworthy: 0
stress-ng: info: [34691] successful run completed in 20.01 secs
20.036485 sec
Pkg_J
681.06

```

6b. Live plot during stress

3.89	0.82	12.25	43.66	4.40	11.56	0.00	0.00	0.00	87.86	3367	51	51	27.14	23.71
0.23	0.00	0.05	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	52	52	32.82	29.43
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	52	52	32.72	29.51
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.76	3600	54	54	33.19	29.65
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.74	3600	54	54	33.68	29.88
0.23	0.00	0.00	0.02	0.00	0.00	0.00	0.00	0.00	99.77	3600	54	54	33.36	29.96
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	55	55	32.83	29.90
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	55	55	33.02	29.92
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	55	55	32.79	29.93
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.76	3600	55	55	32.90	29.82
0.23	0.00	0.00	0.01	0.00	0.00	0.00	0.00	0.00	99.77	3600	55	55	33.03	30.07
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	55	55	33.00	30.02
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	55	55	33.57	30.05
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	56	56	33.42	30.02
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	57	57	32.94	30.00
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	56	56	32.94	30.03
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	57	57	33.06	30.13
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	56	56	33.34	30.17
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	57	57	33.63	30.25
0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.77	3600	57	57	33.75	30.41
1.43	0.31	5.22	12.33	4.54	1.61	0.00	0.00	0.00	35.06	3067	43	43	0.05	6.02



6c. Baseline vs stress energy comparison

Condition	Duration	Pkg_J	Avg PkgWatt	PkgTmp
Baseline	1s	3.3–10.4	3.25–10.10	36–38°C
Stressing	20.01s	649.08	32.42	56°C

stress-ng burns 10× more power than idle baseline. If lab machines sit idle most of the day, **underutilisation wastes energy**. The machine still draws 3–10 W even when idle. At scale, idle servers in data centres waste significant power.

7. matmul energy efficiency comparison

```
ubuntu@ubuntu:~/CWM-FDI$ sudo turbostat -q --Joules --show Pkg_J python3
../assignment1/matmul_slow.py 500
n=500 reps=2 checksum=477458.000000
18.731704 sec
Pkg_J
432.92
432.92

ubuntu@ubuntu:~/CWM-FDI$ sudo turbostat -q --Joules --show Pkg_J python3
../assignment1/matmul_fast.py 500
n=500 reps=2 checksum=477458.000000
13.356689 sec
Pkg_J
333.92
333.92
```

Variant	Time (s)	Energy (J)	Avg Power (W)
matmul_slow	18.73	432.92	23.11
matmul_fast	13.36	333.92	25.00

The fast version consumed 99.00 J less and completed 5.37s faster. The fast code reduces energy because it completes quicker. The reduction in time outweighs the increase in average power.

8. Sleep state observations

The baseline sleep state data shows:

1. **CPU%c7**: 42%
2. **CPU%c6**: 16%
3. **CPU%c1**: 6%
4. **Pkg%pc2**: 18%
5. **Pkg%pc3**: 10%
6. **Pkg%pc6/pc7/pc8**: 0%

No deep package sleep states (pc6 or higher) occurs. The drawback of deeper sleep states is wake-up latency. Deeper C-states take longer to wake up, which can hurt responsiveness for latency-sensitive applications like network packet handling.

9. Utilisation, thermals, and data centre design

High utilization can produce significant heat. Bad thermal will make these heats hard to dissipate out. Several aspect of data centre designs will be affected:

1. Cooling costs will be higher. Cooling requires extra electricity, and cooling devices themselves also generate extra heats.
2. Thermal throttling will limit the performance of devices when heat cannot be sufficiently dissipated.
3. The power delivery network must be designed to sustain high power utilization in peak load periods, even if ordinary power consumption is far below the peak requirements.
4. Accelerators such as GPU clusters demand even more power and cooling than CPU. This leverage up the costs even more.

Network activity

10. Theoretical network energy for 1 GB download

a. Time to transmit 1 GB at 1 Gb/s:

- 1 GB = 8 Gb
- Time = $\frac{8Gb}{1Gb/s} = 8 \text{ seconds}$

b. Energy consumed by the NIC:

- NIC power draw: 0.3–1.0 W (active)
- Energy = $0.3W \times 8s = 2.4 \text{ J}$ (low estimate)
- Energy = $1.0W \times 8s = 8.0 \text{ J}$ (high estimate)

c. End-to-end network energy across multiple hops:

If data travels through multiple routers/switches, each hop adds:

- Per-hop latency
- Per-hop NIC + switch power
- Additional energy for optical transceivers

Assumptions: 10 network hops, each with 1 W active port power. Total network energy $\approx 8 \text{ J} \times 10 = 80 \text{ J}$. A more accurate estimate would require: per-hop power draw, link utilization, packet size distribution, and routing path length.

11. Measured download energy

```
ubuntu@ubuntu:~/CWM-FDI$ sudo turbostat -q --Joules --show Pkg_J wget -O
/tmp/sg_90k_part1.json
"https://huggingface.co/datasets/RyokoAI/ShareGPT52K/resolve/main/sg_90k_par
t1.json?download=true"
...
2026-06-09 11:00:11 (55.4 MB/s) - '/tmp/sg_90k_part1.json' saved
[921586083/921586083]
16.171105 sec
Pkg_J
393.71
393.71
```

Metric	Theoretical	Measured
File size	1 GB	921 MB
Time	8 s	16.17 s
Throughput	1000 Mb/s	445 Mb/s (55.4 MB/s)
Energy	2.4–8 J (NIC only)	393.71 J

The measured results differ significantly from the theoretical estimate because:

- Theoretical only considered NIC power while measured includes full system including CPU, memory, network interface etc.
- Actual throughput was below line rate due to server-side limits and network congestion.
- Total system power during download was 24.3 W (393.71 J / 16.17 s), far more than the NIC alone.

12. Video streaming energy

Resolution	PkgWatt	Observations
480p (SD)	~5-6 W	Minimal GPU/CPU decode load
1080p (HD)	~6–8 W	Moderate decode load
2160p (4K)	~8–12 W	Heavy GPU/CPU decode load

Higher resolutions consume more energy due to increased GPU/CPU decode workload.

Energy efficiency of video streaming depends on hardware video decoding. Modern GPUs with dedicated media engines decode high resolution videos at far lower energy than software decoding.

Thought Experiments – Carbon Footprint

13. Carbon footprint of matmul variants

Carbon Intensity values used:

Location	CI (gCO ₂ e/kWh)	Source
London, UK	77	NESO Carbon Intensity API (2026-06-09)
France (national avg)	20	RTE (2025–2026 avg: ~20 gCO ₂ e/kWh)

Energy consumed (from Q7):

Variant	Energy (J)	Energy (kWh)
matmul_slow (n=500)	432.92	0.0001203
matmul_fast (n=500)	333.92	0.00009276

Carbon footprint (CFP = E × CI):

Location	CI (g/kWh)	Slow CFP (gCO ₂)	Fast CFP (gCO ₂)	Savings
London, UK	77	0.00926	0.00714	22.9%
France	20	0.00241	0.00186	22.9%

The fast version saves 22.9% carbon emissions compared with slow version.

At scale, AI inference performs billions of matrix multiplications daily. Even per-operation savings multiply enormously. For example, if AI inference consumes 10 GWh/day globally, a 22.9% optimisation saves 2.29 GWh/day, equivalent to 176 tonnes CO₂/day (at UK CI).

To minimise carbon footprint, code should be run when and where CI is lowest, e.g. during periods of high renewable generation and in regions with clean grids.

14. ChatGPT's carbon footprint

Given:

- 0.34 Wh per standard text query (Sam Altman, June 2025)
- 2.5 billion prompts/day (Axios)

Daily energy:

- $0.34 \text{ Wh} \times 2.5 \times 10^9 = \mathbf{850,000,000 \text{ Wh} = 850 \text{ MWh/day}}$

Carbon footprint (at UK 77 gCO₂/kWh):

- Daily: $850 \text{ MWh} \times 77 \text{ g/kWh} = 65,450 \text{ kg CO}_2 = \mathbf{65.45 \text{ tonnes CO}_2/\text{day}}$
- Monthly: **~1,964 tonnes CO₂/month**
- Yearly: **~23,890 tonnes CO₂/year**

Comparison with training emissions:

Model	Estimated Training CFP	Source
GPT-3	~552 tCO ₂	Patterson et al. (2021)
GPT-4	~2,300–5,000 tCO ₂	Public estimates
ChatGPT daily inference	65.45 tCO ₂	Calculated above

Inference dominates. ChatGPT's daily inference (65 tCO₂) exceeds GPT-3's entire training cost (552 tCO₂) in about 8.4 days. Annual inference (~23,890 tCO₂) is 4–10× larger than GPT-4 training.

Perspective:

- A lab of 100 desktop machines (each 150W, 24/7): ~132 tCO₂/year (at UK CI)
- A typical UK home: ~4 tCO₂/year
- Global aviation: ~915 MtCO₂/year
- ChatGPT annual inference: ~23,890 tCO₂ **≈ 5,972 UK homes' annual electricity or 12% of a typical gas-fired power plant's daily output**

Sources:

- Sam Altman, "The Gentle Singularity" (June 2025)
- Axios: ChatGPT prompt volume estimates
- NESO Carbon Intensity API (<https://api.carbonintensity.org.uk/>)
- Patterson et al., "Carbon Emissions and Large Neural Network Training" (2021), arXiv:2104.10350
- RTE France (<https://analysesetdonnees.rte-france.com>)